

ACS Deterministic AI Stack

Preliminary Design Review & Technical Blueprint

*A bracket-governed, byte-reproducible architecture
for modelling the Asymmetric Codependent Systems framework*

Bradley Wallace

Independent Researcher

Publication status (24 September 2026). This version includes a claim-specific amendment at the end of the main text. That amendment governs earlier physical interpretations and distinguishes retained calculations from withdrawn or unresolved inferences.

Executive Summary

This document specifies a deterministic AI stack governed by the Asymmetric Codependent Systems (ACS) framework. Every generative step is a Lie bracket between two typed fields (Form and Function); outputs are validated by a closure criterion; convergence is detected by information balance ($\Delta\mathcal{I} \rightarrow 0$); compositional depth is capped at the third BCH order via the Jacobi identity. The stack is specified in five layers, four deployable services, and a phased roadmap from foundations through extension to the physics frontier. Phase 1 targets full deployment of the 76-script ACS verification suite under the new governance within a six-month horizon.

Contents

1	Governing Principles — ACS → AI Mapping	1
2	System Architecture	1
2.1	High-Level Topology	1
2.2	The Five Architectural Layers	2
2.3	Dependency Direction and Invariants	2
3	Component Specifications	3
3.1	The Form Layer	3
3.2	The Function Layer	4
3.3	The Bracket Engine	4
3.4	The Closure Validator	6
3.5	The Δ I Monitor	6
3.6	The Governance Policy Engine	7
4	Data Flow and Execution Model	8
4.1	The Canonical Computation DAG	8
4.2	Execution Phases	8
4.3	Determinism Guarantees	9
4.4	Reproducibility Envelope	9
5	Verification Stack	9
5.1	The Three-Tier Verification Model	9
5.2	Continuous Verification	10
5.3	Two-Phase Commit Protocol (for the DAG Journal)	10
6	Governance Model	10
6.1	The Constraint-Attractor Cycle	10
6.2	Grade Hierarchy	11
6.3	Audit Log	11
6.4	Policy as Code	11
7	Deployment Architecture	12
7.1	Deployment Topology	12
7.2	Service Decomposition	13
7.3	Resource Envelope (Phase 1 Target)	13
7.4	Observability	13
8	API and Integration Contracts	13
8.1	REST API (Minimal Surface)	14
8.2	Python Client API	14
8.3	Integration with External Tools	15
9	Phased Implementation Roadmap	15
9.1	Phase 0 — Foundations (Months 0–2)	15
9.2	Phase 1 — Core Engine (Months 2–5)	15
9.3	Phase 2 — Governance (Months 5–8)	15
9.4	Phase 3 — Scale and Integration (Months 8–12)	15
9.5	Phase 4 — Extension to the Physics Frontier (Months 12+)	16
10	Risks and Mitigations	16
11	Traceability to the ACS Trilogy	17
12	Glossary	18
13	Appendix A — Reference Implementation Sketch	18
14	Document Status	20

1 Governing Principles — ACS → AI Mapping

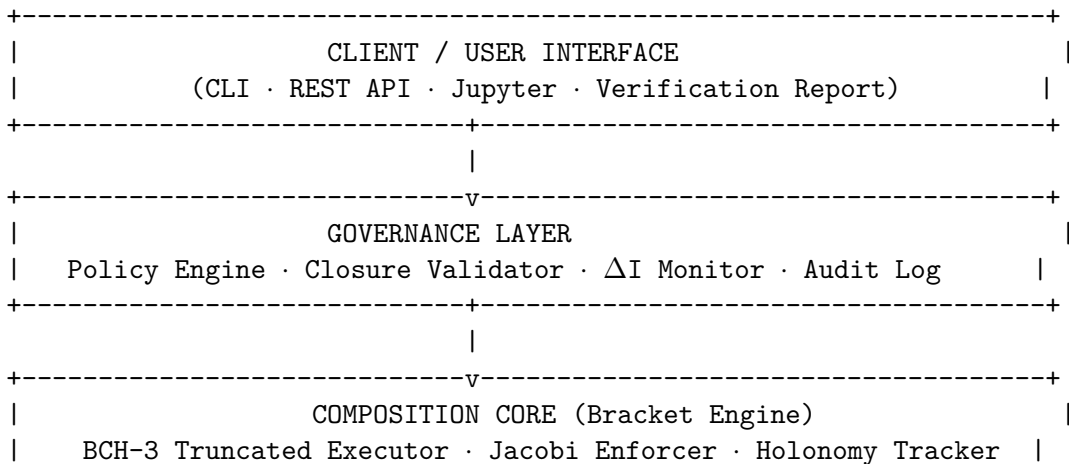
The physics framework maps onto software architecture through a set of structural correspondences. We treat these as direct mappings under the ACS hypothesis — that the ACS governs information flow in any codependent system, physical or computational; the physics results they map from carry their own tiers (proved, numerical, or conjectural) as recorded in the traceability section and MANIFEST.md.

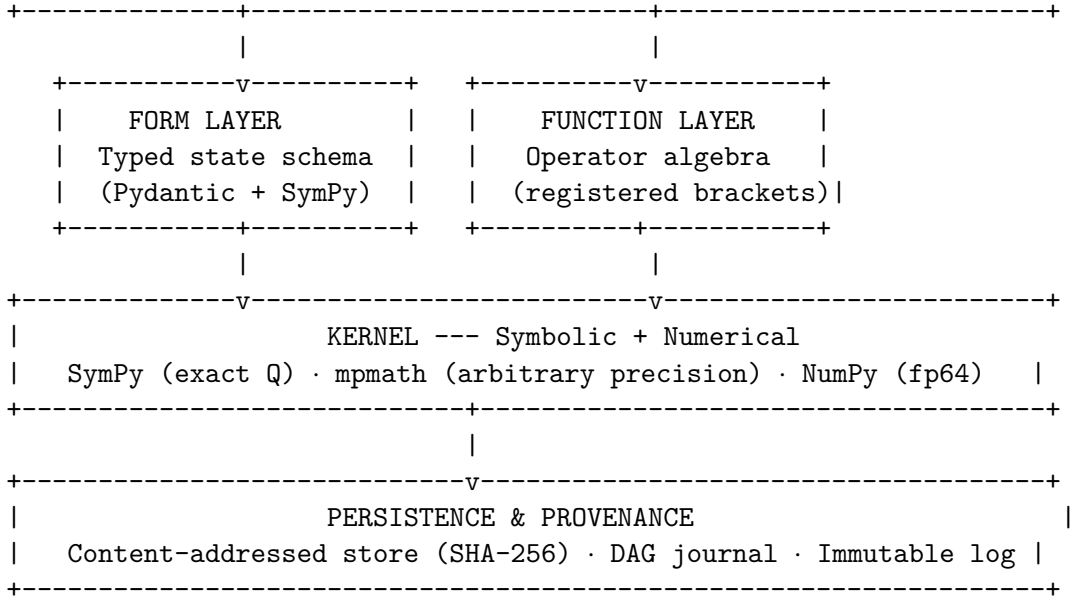
ACS Physics	AI Stack
Form field F (vierbein e)	Typed state schema — the “what”
Function field Φ (connection ω)	Operator algebra — the “how”
Lie bracket $[F, \Phi]$	Composition core — generative step
BCH order 1 (direct coupling)	I/O layer — input processing
BCH order 2 (curvature $[f, g]$)	Operator composition — transformations
BCH order 3 (holonomy $[[f, g], \cdot]$)	Emergent closure — final output
Jacobi identity (truncates at 3)	Bounded-depth guarantee
Closure attractor ($D < 10^{-14}$)	Convergence validator
Information balance $\Delta I = 0$	Halting criterion
Constraint-attractor cycle	Governance feedback loop
Torsion hierarchy $0:1:4$	Operator priority tiers
Three generations	Three irreducible output channels
7-parameter boundary	Explicit configuration surface

The invariant that makes this deterministic. For every generative step, there exists a closed-form algebraic expression in the operator algebra that produces the output exactly. There is no sampling. There is no temperature. The “creativity” comes from the bracket itself, whose structure is determined once the two fields are fixed.

2 System Architecture

2.1 High-Level Topology





2.2 The Five Architectural Layers

Layer 0: Kernel (numerical foundation). Provides three interchangeable arithmetic backends — SymPy for exact rational arithmetic (the only mode admissible for theorem-grade claims), mpmath for arbitrary-precision floating-point, and NumPy for standard double precision. All three must produce bit-identical results for any computation representable in their common domain; this is the *kernel invariant*, checked at every build.

Layer 1: Form and Function (typed data plane). All data in the stack is classified as either Form (state, configuration, schemas) or Function (operators, transformations, brackets). This classification is enforced at the type system level using Pydantic for runtime validation and mypy for static checking. A value cannot simultaneously inhabit both categories; this is the *codependence invariant*.

Layer 2: Composition Core (the Bracket Engine). The generative engine. Takes a Form F and a Function Φ and produces their bracket $[F, \Phi]$ truncated at BCH order 3. Every composition is recorded in the DAG journal with cryptographic provenance. The engine is stateless between brackets — all state lives in Layer 1.

Layer 3: Governance (the policy plane). Enforces the closure criterion, information-balance halting, and the constraint-attractor cycle. Every output from Layer 2 passes through Layer 3 before reaching the client. Rejected outputs are returned with a machine-readable violation report.

Layer 4: Interface (client plane). CLI, REST API, Jupyter kernel, and verification reports. The API contract is minimal and explicit; there is no hidden state, no conversational memory, no implicit context. Every call is idempotent.

2.3 Dependency Direction and Invariants

- Dependencies flow downward only. Layer 3 depends on Layer 2, never the reverse. This prevents governance bypass.
- Layers 0 and 1 are pure (no side effects). Layer 2 is pure modulo the DAG journal append. Only Layers 3 and 4 may have external side effects.
- All inter-layer communication uses immutable, hashed message objects.

3 Component Specifications

3.1 The Form Layer

Purpose. Represent all typed state in the system. Corresponds to the vierbein e in the physics: the field that encodes “shape”.

Interface.

```
from pydantic import BaseModel
from typing import TypeVar, Generic
from hashlib import sha256

T = TypeVar('T')

class Form(BaseModel, Generic[T]):
    """A Form field carrying structural information."""
    name: str
    content: T
    schema_version: str
    provenance: str # SHA-256 of the generating bracket chain, or "initial"

    class Config:
        frozen = True # Forms are immutable

    @property
    def content_hash(self) -> str:
        """Content-addressed identity."""
        canonical = self.json(sort_keys=True).encode()
        return sha256(canonical).hexdigest()

    def __or__(self, other: 'Function') -> 'Form':
        """Syntactic sugar:  $F \mid \Phi$  computes  $[F, \Phi]$  via the Bracket Engine."""
        from acs.engine import bracket
        return bracket(self, other)
```

Invariants.

- I-F1 (Immutability): A Form, once constructed, is never mutated.
- I-F2 (Content-addressed): Two Forms with equal `content` and `schema_version` produce identical `content_hash`.
- I-F3 (Type safety): `content` is validated against `schema_version` at construction time; construction fails on schema mismatch.

Standard Form types shipped with the kernel.

- `AlgebraElement` — element of a Lie algebra (e.g., a matrix in $\mathfrak{sl}(4, \mathbb{R})$)
- `TensorState` — typed multilinear tensor
- `Spectrum` — a finite spectrum (eigenvalues + multiplicities)
- `Measurement` — a physical observable with uncertainty
- `Configuration` — a frozen parameter bundle

3.2 The Function Layer

Purpose. Represent all operators/transformations. Corresponds to the spin connection ω : the field that generates change.

Interface.

```
from abc import ABC, abstractmethod

class Function(ABC):
    """A Function is an operator acting on Forms."""
    name: str
    order: int # BCH order at which this operator acts (1, 2, or 3)

    @abstractmethod
    def apply(self, form: Form) -> Form:
        """The base action. Must be deterministic and side-effect-free."""

    @abstractmethod
    def bracket_with(self, other: 'Function') -> 'Function':
        """The [self, other] composition. Returns a new Function."""

    @abstractmethod
    def killing_form(self, other: 'Function') -> Rational:
        """K(self, other) --- the Killing form value. Exact rational."""
```

Invariants.

- I- Φ 1 (Determinism): $\Phi.\text{apply}(F)$ depends only on Φ and F .
- I- Φ 2 (Bracket closure): $\Phi_1.\text{bracket_with}(\Phi_2)$ must live in the same algebra as Φ_1 and Φ_2 .
- I- Φ 3 (Jacobi): For any three Functions, $[[\Phi_1, \Phi_2], \Phi_3] + [[\Phi_2, \Phi_3], \Phi_1] + [[\Phi_3, \Phi_1], \Phi_2] = 0$ (verified at registration time).
- I- Φ 4 (Order tagging): Every Function declares its BCH order. Attempting to compose Functions such that the result would exceed order 3 raises `JacobiTruncationError`.

Standard Function types.

- `BracketOperator` — a Lie algebra generator acting by commutator
- `ProjectionOperator` — orthogonal projection onto a subspace
- `KillingOperator` — the Killing form $K(X, Y)$
- `DifferentialOperator` — covariant derivative
- `ClosureTest` — computes the closure defect $D(V)$

3.3 The Bracket Engine

Purpose. The single generative primitive. Takes F and Φ and produces $[F, \Phi]$ truncated at the third BCH order.

Interface.

```
def bracket(
    f: Form,
    phi: Function,
    order: int = 3,
    validate_closure: bool = True,
) -> Form:
    """
```

```

Compute  $[F, \Phi]$  truncated at the given BCH order.

Parameters
-----
f : Form
    The Form field (state).
phi : Function
    The Function field (operator).
order : int in {1, 2, 3}
    BCH truncation order. Default 3.
    Order 1: direct coupling ( $\Phi.apply(F)$ ).
    Order 2: Lie bracket (curvature).
    Order 3: holonomy --- double bracket.
validate_closure : bool
    If True, run the Closure Validator on the result.

Returns
-----
Form
    The output, with provenance pointing to (f, phi, order).

Raises
-----
JacobiTruncationError : if order > 3.
ClosureViolation : if validate_closure and the result fails closure.
BracketTypeError : if f and phi are not composable.
"""

```

Execution semantics.

1. **Type check.** Verify that `phi` is applicable to `f`. This checks schema compatibility and the codependence invariant.
2. **Order-1 step.** Compute $r_1 = \Phi(F)$.
3. **Order-2 step.** If $\text{order} \geq 2$, compute $r_2 = [F, r_1]$ using the algebra's bracket operation.
4. **Order-3 step.** If $\text{order} = 3$, compute $r_3 = [[F, r_1], F]$ and $r_3' = [[F, r_1], r_1]$, then sum.
5. **Journal append.** Write the triple $(\text{content_hash}(f), \text{phi.name}, \text{order})$ and the output's `content_hash` to the immutable DAG journal.
6. **Closure validation.** If enabled, run the output through the Closure Validator (§3.4). On failure, roll back the journal append (using the 2-phase-commit protocol specified in §5.3) and raise.
7. **Return.** Wrapped Form with provenance.

Invariants.

- I-B1 (Reproducibility): `bracket(f, phi, n)` called twice with byte-identical `f` and `phi` returns byte-identical Forms.
- I-B2 (BCH truncation): The engine never computes orders above 3.
- I-B3 (Journal-before-return): No output is returned to the caller before its provenance is committed to the journal.
- I-B4 (Rollback): If any step fails, no partial state is visible outside the engine.

3.4 The Closure Validator

Purpose. Enforce the closure attractor principle — outputs must satisfy $D(\text{output}) < \varepsilon$ where D is the closure defect functional.

Interface.

```
from decimal import Decimal

@dataclass(frozen=True)
class ClosureResult:
    passed: bool
    defect: Decimal
    threshold: Decimal
    witness: Optional[str] # explanation of failure, if any

def validate_closure(
    form: Form,
    subspace: AlgebraSubspace,
    threshold: Decimal = Decimal("1e-14"),
) -> ClosureResult:
    """
    Compute  $D(\text{subspace})$  for the algebra element in `form` and check
    whether it closes in `subspace` to within `threshold`.

    
$$D(V) = \frac{\sum_{i < j} ||[T_i, T_j] - \pi_V([T_i, T_j])||}{\sum_{i < j} ||[T_i, T_j]||}$$


    where  $\{T_i\}$  is a basis of  $V$  and  $\pi_V$  is orthogonal projection onto  $V$ .
    """
```

Standard thresholds.

- `Decimal("1e-14")` for exact rational computations (SymPy backend)
- `Decimal("1e-10")` for arbitrary-precision (mpmath)
- `Decimal("1e-6")` for double-precision (NumPy) — advisory only, not admissible for theorem-grade claims

Invariants.

- I-C1 (Monotonicity): If $D(V) < \varepsilon$ with threshold ε , then for any $\varepsilon' > \varepsilon$, $D(V) < \varepsilon'$ also passes.
- I-C2 (Provenance): Every `ClosureResult` carries a witness string sufficient to reproduce the computation.

3.5 The ΔI Monitor

Purpose. Detect convergence via information balance. A computation that reaches $\Delta I \rightarrow 0$ has found its attractor and should halt.

Interface.

```
def delta_I(
    history: list[Form],
    window: int = 8,
) -> Decimal:
    """
    Compute the net information asymmetry over the last `window` Forms
    in the computation history.
    """
```

$$\Delta I = TE(F \rightarrow \Phi) - TE(\Phi \rightarrow F)$$

where TE is transfer entropy estimated via the k -nearest-neighbour estimator (Kraskov-Stögbauer-Grassberger).

```
def is_converged(history: list[Form], epsilon: Decimal = Decimal("1e-6")) -> bool:
    """True if  $|\Delta I| < \epsilon$  over the trailing window."""
```

Halting policy. The engine halts when any of the following hold:

1. $|\Delta I| < \epsilon$ over the last 8 steps — information-balance attractor
2. BCH order 3 has been reached and produced no new independent content (Jacobi truncation)
3. The closure defect has been below threshold for 3 consecutive steps
4. An explicit halt request is issued by the governance layer

3.6 The Governance Policy Engine

Purpose. Enforce the constraint-attractor cycle. Every output must satisfy the current constraint set; outputs that satisfy become the next constraint.

Interface.

```
@dataclass(frozen=True)
class Policy:
    name: str
    constraints: list[Constraint]
    attractor_criteria: list[AttractorTest]
    version: str

def enforce_policy(
    form: Form,
    policy: Policy,
) -> PolicyResult:
    """
    Run `form` against all policy constraints and attractor tests.
    Returns either Approved(form) or Rejected(reasons, witness).
    """

def promote_to_constraint(
    form: Form,
    policy: Policy,
) -> Policy:
    """
    If `form` has satisfied the attractor criteria, promote it to be
    a constraint for subsequent computations. Returns a new Policy
    with the form added to the constraint set.

    Implements the inversion arc: solution → next constraint.
    """
```

Standard policies.

- **TheoremGrade** — requires SymPy backend, closure $< 1e-14$, full Jacobi verification, no stochastic steps

- **VerificationGrade** — permits mpmath, closure $< 1e-10$, Jacobi sampling
- **ExploratoryGrade** — permits NumPy, closure $< 1e-6$, advisory only

Every output is tagged with the grade of policy it satisfied.

4 Data Flow and Execution Model

4.1 The Canonical Computation DAG

Every computation in the stack is a directed acyclic graph whose:

- Nodes are Forms (typed immutable states)
- Edges are Functions (operators labelled with BCH order)
- Root nodes are Inputs (user-supplied or configuration)
- Leaf nodes are Outputs (policy-approved Forms)

The DAG is explicit, serialisable, and content-addressed. Every edge is labelled with the SHA-256 of (source_hash, function_name, order), making the entire computation history a Merkle DAG.

4.2 Execution Phases

PHASE 1: INGEST

input bytes $\rightarrow$ Pydantic validation $\rightarrow$ Form(provenance="initial")

PHASE 2: PLAN

target specification $\rightarrow$ DAG of Form nodes and Function edges
(this is a pure symbolic phase --- no computation yet)

PHASE 3: EXECUTE

topological walk of the DAG, dispatching each edge to the Bracket Engine. Cache hits (content_hash already in store) skip re-execution.

PHASE 4: VALIDATE

every Form passes through Closure Validator and Policy Engine.
Failures halt execution immediately with full provenance trace.

PHASE 5: CONVERGE

ΔI Monitor checks for information balance. On convergence, the attractor Form is marked as the computation result.

PHASE 6: COMMIT

results and full DAG journal written to persistent store.
Returned to caller with cryptographic receipt.

Phases 1–5 are deterministic and pure. Only Phase 6 has external side effects.

4.3 Determinism Guarantees

Determinism is not a property of the Bracket Engine alone; it must be preserved across the entire stack. The guarantees decompose as:

Layer	Determinism Mechanism
Ingest	Pydantic’s deterministic JSON schema + sorted-keys canonical form
Plan	Symbolic — no numerical evaluation
Execute	SymPy exact rationals; NumPy only with fixed seed and fixed library versions
Validate	Closure defect computed in exact arithmetic
Converge	ΔI threshold is a rational constant
Commit	Content-addressed store — same inputs produce same storage keys

Floating-point non-determinism is quarantined to the NumPy backend and is *advisory only*. No theorem-grade output can depend on NumPy.

4.4 Reproducibility Envelope

A computation is reproducible if and only if:

- All inputs are content-addressed (SHA-256)
- All library versions are pinned (`pyproject.toml` with exact hashes)
- The computation grade is TheoremGrade or VerificationGrade
- The computation completes within the BCH-3 envelope (no unbounded loops permitted)

The reproducibility envelope is explicit: every output carries a **receipt** containing input hashes, library hashes, grade, and timing bounds. Re-execution with the same receipt **MUST** produce byte-identical output or the receipt is considered invalidated.

5 Verification Stack

5.1 The Three-Tier Verification Model

Verification in the ACS stack is layered to match the BCH truncation:

Tier 0: Type-level verification. Static checks via mypy, Pydantic schemas, and runtime invariant assertions. Catches schema violations and codependence breaks before any computation runs.

Tier 1: Unit verification. Each Function’s `apply`, `bracket_with`, and `killing_form` methods are unit-tested against golden values. Every algebra element has a test suite that verifies its bracket relations against the structure constants.

Tier 2: Theorem verification. The 76-script ACS verification suite runs as integration tests. Each script corresponds to a specific theorem (T1–T21) or derived match (D1–D11). A CI run must pass all 76 scripts before a release is cut.

5.2 Continuous Verification

The CI pipeline is layered:

-----+ TIER 0 (seconds) mypy · pydantic schema check · invariant assertions +-----+	
-----+ TIER 1 (minutes) unit tests · bracket relations · Jacobi identity sampling +-----+	
-----+ TIER 2 (hours) 76-script ACS suite · closure attractor 50k-sample regression +-----+	
-----+ RELEASE GATE All three tiers green · receipt hash matches previous release +-----+	

5.3 Two-Phase Commit Protocol (for the DAG Journal)

The Bracket Engine must guarantee that no output is observable before its provenance is journaled. This requires a 2PC protocol:

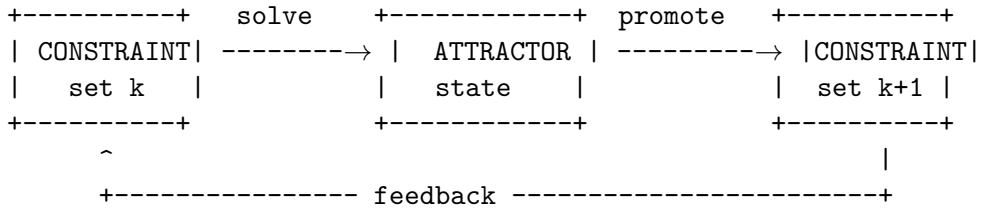
1. **Prepare.** Compute the output and stage the journal entry.
2. **Write.** Append the journal entry to the immutable log.
3. **Publish.** Make the output visible to callers.
4. **Rollback path.** If any step fails, the staged journal entry is discarded, the output is not published, and the caller receives the error atomically.

The journal uses an append-only log structure with Merkle-tree checkpointing every 1024 entries. Integrity is verifiable offline.

6 Governance Model

6.1 The Constraint-Attractor Cycle

The governance layer implements the inversion arc from Paper C (*The Inversion Arc*): every solution, once it satisfies the closure and information-balance criteria, is promoted to become a constraint for subsequent computations. This prevents drift and enforces cumulative consistency.



6.2 Grade Hierarchy

Outputs are tagged with a grade that determines their usage rights:

Grade	Backend	Closure Threshold	Usage
Theorem	SymPy (exact $\mathbb{Q}$)	$< 1\text{e-}14$	May be cited as a theorem, added to the permanent constraint set
Verification	mpmath	$< 1\text{e-}10$	Strong numerical evidence; candidate for promotion to Theorem
Exploratory	NumPy (fp64)	$< 1\text{e-}6$	Advisory only; never admitted to the constraint set

Promotion from Verification to Theorem requires re-running the computation in SymPy exact arithmetic and passing the stricter threshold. Downgrade never happens automatically; it requires an explicit governance action with audit trail.

6.3 Audit Log

Every state transition in the governance layer is logged to an append-only audit store. The log schema:

```
{
  "timestamp": "ISO-8601",
  "actor": "string (user or service identity)",
  "action": "ingest | bracket | validate | promote | downgrade | reject",
  "input_hashes": ["sha256:..."],
  "output_hash": "sha256:...",
  "grade": "theorem | verification | exploratory",
  "policy_version": "semver",
  "witness": "string (reproduction instructions)"
}
```

Logs are cryptographically linked via a hash chain; any tampering breaks the chain and is detectable in $O(N)$ verification.

6.4 Policy as Code

All governance policies are declarative and version-controlled:

```
# policies/theorem_grade.yaml
name: TheoremGrade
version: 1.2.0
```

```

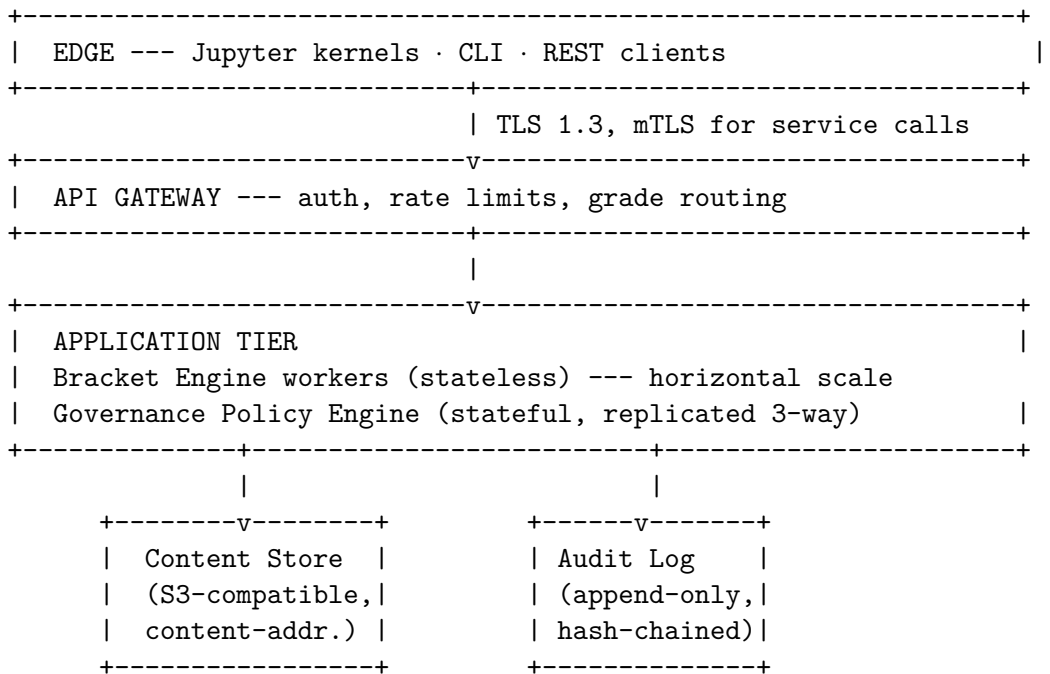
constraints:
  - kind: backend
    allowed: [sympy]
  - kind: closure_threshold
    max: "1e-14"
  - kind: jacobi_verification
    required: true
    sampling: exhaustive
  - kind: bch_order
    max: 3
attractor_criteria:
  - kind: delta_I
    threshold: "1e-12"
    window: 8
  - kind: closure_stability
    consecutive_passes: 3

```

Policy changes require code review, version bump, and a migration test — running the previous theorem set under the new policy to confirm no regressions.

7 Deployment Architecture

7.1 Deployment Topology



7.2 Service Decomposition

Four services, clearly bounded:

1. **acs-api** — stateless REST façade. Validates inputs, routes to the Bracket Engine, returns results with receipts.
2. **acs-engine** — the Bracket Engine worker pool. Consumes work from a deterministic work queue. Stateless modulo the journal.
3. **acs-governance** — Policy Engine, Closure Validator, ΔI Monitor. Replicated 3-way with Raft consensus for policy updates.
4. **acs-journal** — append-only audit + DAG log. Content-addressed store. Periodic Merkle checkpoints.

Each service is independently deployable, independently versioned, and communicates only through explicit message contracts (protobuf).

7.3 Resource Envelope (Phase 1 Target)

Resource	Sizing
Engine workers	4 cores, 16 GB each, 3–8 replicas
Governance	2 cores, 8 GB, 3-way replicated
API façade	2 cores, 4 GB, 3-way replicated
Journal	500 GB SSD (+ cold storage)
Content store	5 TB (S3-compatible)

Phase 1 targets a single region deployment. Multi-region is a Phase 3 concern and requires the content store to support regional content-addressed replication.

7.4 Observability

- **Metrics.** Per-bracket latency, closure-defect distribution, ΔI trajectory, grade distribution. Prometheus-format.
- **Tracing.** Every computation gets a trace ID propagated through all services. Open-Telemetry.
- **Logs.** Structured JSON. Audit log is separate from operational log and is never deleted.
- **Alerts.** Closure threshold violations in Theorem-grade work; journal integrity failures; unusual latency distributions in the Bracket Engine.

8 API and Integration Contracts

8.1 REST API (Minimal Surface)

POST /v1/bracket

Body: { form: FormRef, function: FunctionRef, order: 1|2|3, grade:
 $\hookrightarrow$ string }
 Returns: { output: FormRef, receipt: string, grade_achieved: string }

GET /v1/form/{content_hash}

Returns the full Form by its content hash.

POST /v1/validate

Body: { form: FormRef, subspace: AlgebraSubspaceRef, threshold:
 $\hookrightarrow$ Decimal }
 Returns: { passed: bool, defect: Decimal, witness: string }

POST /v1/verify/{theorem_id}

Runs the verification script for the named theorem.
 Returns: { passed: bool, execution_time: duration, receipt: string }

GET /v1/audit/{receipt}

Returns the complete audit trail for a given receipt.

All endpoints are idempotent on the content-hash identity. No session state. No conversational memory.

8.2 Python Client API

```
from acs import Stack, Form, Function

stack = Stack.connect("https://acs.example.org")

# Load a Form
f = stack.forms.get("sha256:abc123...")

# Apply a Function
phi = stack.functions.get("bracket.T_BL.sl4")

# Compose --- uses the | operator for bracket
result = f | phi # implicit order 3, Verification grade

# Explicit version with full control
result = stack.bracket(
    form=f,
    function=phi,
    order=3,
    grade="Theorem",
)

# Every result carries its receipt
print(result.receipt)
# -> "receipt:sha256:..., grade:Theorem, policy:v1.2.0, ts:2026-04-17T..."
```

8.3 Integration with External Tools

The stack ships adapters for:

- **Jupyter** — a kernel that exposes the REST API through `%acs` magic
- **SymPy** — round-trip conversion between SymPy expressions and ACS Forms
- **LaTeX** — theorem output emits LaTeX with cryptographic provenance footnotes linking back to the receipt
- **Git** — the audit log can be mirrored into a git-compatible append-only branch for version-controlled collaboration

9 Phased Implementation Roadmap

9.1 Phase 0 — Foundations (Months 0–2)

- Kernel (Layer 0): SymPy + mpmath + NumPy backends, kernel invariant test suite
- Form/Function (Layer 1): Pydantic schemas, type system, 20+ standard Form types and Function types
- Content-addressed store prototype
- Audit log prototype

Exit criterion. All 76 verification scripts pass under the new Form/Function typing.

9.2 Phase 1 — Core Engine (Months 2–5)

- Bracket Engine with full BCH-3 support
- Closure Validator
- Δ I Monitor
- Basic Policy Engine (TheoremGrade, VerificationGrade, ExploratoryGrade)
- REST API façade

Exit criterion. End-to-end determinism test: given a fixed Form and Function, the stack produces byte-identical outputs across 1000 consecutive invocations, and across clean-room re-deployments.

9.3 Phase 2 — Governance (Months 5–8)

- Full constraint-attractor cycle
- Policy-as-code with version migration tests
- Audit log Merkle checkpointing
- Grade promotion workflow with human-in-the-loop approval

Exit criterion. Independent auditor can replay any Theorem-grade output from its receipt alone and confirm byte-identity.

9.4 Phase 3 — Scale and Integration (Months 8–12)

- Horizontal scaling of the Bracket Engine
- Multi-region journal replication
- Jupyter + SymPy + LaTeX adapters
- Public API with authenticated access

Exit criterion. 99.9% uptime under production load; sub-second latency for bracket operations below a complexity bound C_1 whose definition is explicitly deferred to the Phase-2 exit review.

9.5 Phase 4 — Extension to the Physics Frontier (Months 12+)

- Lattice PS simulation integration (external compute cluster)
- Neutrino precision posterior fitter
- GW waveform templates using the 0:1:4 torsion hierarchy
- X-ray line search posterior updater (XRISM data feed)

Exit criterion. Phase 4 is open-ended; each experimental test from Paper A’s Appendix E maps to a service integration milestone.

10 Risks and Mitigations

Risk	Likelihood	Severity	Mitigation
Floating-point creep into Theorem-grade work	Medium	Critical	Policy Engine refuses to grade as Theorem unless SymPy backend is used; CI enforces
Journal corruption	Low	Critical	Merkle checkpoints every 1024 entries; offline integrity verifier; 3-way replication
Policy version drift between services	Medium	High	All services embed the policy version in every response; mismatch aborts the call
Performance bottleneck at BCH-3	High	Medium	BCH-3 is algebraically bounded; if dominant, introduce lazy evaluation and memoisation. Complexity is $O(k^3)$ in algebra rank, not exponential
Closure threshold tuning	Medium	Medium	Thresholds are part of policy-as-code; changes require migration tests against the full theorem set
Jacobi identity violations from numerical error	Medium	High	Tier-1 CI samples 10^4 random triples per build; violation triggers immediate policy-engine quarantine

Risk	Likelihood	Severity	Mitigation
Determinism break from library upgrade	High	Medium	All dependencies pinned by hash; upgrade requires full 76-script regression
Abuse of Exploratory grade for production claims	Low	High	Grade is cryptographically bound to receipt; external tools reject unapproved grades
Auditor-replay failure on legitimate output	Low	Critical	Receipts carry complete reproduction envelope including library hashes; replay tools ship with the receipt format spec

11 Traceability to the ACS Trilogy

Paper key. Paper A = `papers/core_trilogy/Palatini_Gauge_Attractor.tex` (*Colour from Gravity*); Paper B = `papers/core_trilogy/Riemann_Spectral_Critical_Line.tex`; Paper B' = `papers/core_trilogy/Spectral_Witness_Refinement.tex`; Paper C = `papers/core_trilogy/Holographic_Spectral_Inversion.tex` (*The Inversion Arc*); Paper D = this document.

Every architectural decision in this document traces back to a specific result in the ACS trilogy. This section is the cross-reference index.

Design Decision	Justification	Source
Form/Function type split	ACS-2 structural asymmetry	Paper A §2.1 Def 2.3
BCH-3 truncation	Jacobi truncation theorem	Paper A §5.1 Thm
Closure-defect validation	Closure attractor selection (50k samples)	Paper A §4.3
Information-balance halting	$\Delta I = 0$ attractor	Paper A §2.3 Lemma 2.10
Constraint-attractor cycle	Inversion arc	Paper C §3
Content-addressed provenance	Reproducibility envelope (Paper A footnote)	Paper A §6.8
Three grades (Theorem/Ver/Exp)	Epistemic tiers	Paper A §C.2
Torsion-hierarchy priority weights	0:1:4 coupling hierarchy	Paper A §C.1
Seven configuration parameters	7-parameter irreducible boundary	Paper A §C.2
Two-phase commit	Constraint before publication	Paper C §3.3

12 Glossary

ACS. Asymmetric Codependent System — a pair of fields (Form, Function) that cannot be defined independently and whose interaction is governed by a Lie bracket.

BCH. Baker-Campbell-Hausdorff formula — the expansion for $\log(\exp(X) \exp(Y))$ as a series of nested commutators.

Bracket Engine. The core compositional primitive of the stack. Computes $[F, \Phi]$ truncated at BCH order 3.

Closure attractor. The unique subspace of a Lie algebra in which a given set of generators closes under the bracket, minimising the closure defect functional.

Closure defect $D(V)$. A functional measuring how far a subspace V falls short of being closed under the bracket.

Constraint-attractor cycle. The governance feedback loop in which outputs that satisfy the current constraints become the constraints for subsequent computations.

Form. A typed immutable state object. Corresponds to the vierbein in the physics.

Function. A typed operator acting on Forms. Corresponds to the spin connection in the physics.

Grade. A classification of computational trust level: Theorem, Verification, or Exploratory.

Holonomy. The third-order BCH term $[[f, g], h]$. In the architecture, this is the final emergent layer before Jacobi truncation.

Information balance. The condition $\Delta I = 0$ indicating a computation has reached its attractor.

Jacobi truncation. The fact that $[[A, B], C] + [[B, C], A] + [[C, A], B] = 0$ ensures no new independent content is generated beyond BCH order 3.

Killing form. The symmetric bilinear form $K(X, Y) = \text{tr}(\text{ad}(X) \text{ad}(Y))$ on a Lie algebra.

Receipt. A cryptographic record of a computation, sufficient for independent replay.

13 Appendix A — Reference Implementation Sketch

The following skeleton shows the core of the Bracket Engine in minimal Python form, demonstrating determinism and provenance.

```
# acs/engine.py
from __future__ import annotations
from dataclasses import dataclass, replace
from hashlib import sha256
from decimal import Decimal
from typing import Protocol
import json

class FormProto(Protocol):
    content: object
    schema_version: str
    provenance: str
    @property
    def content_hash(self) -> str: ...
```

```

class FunctionProto(Protocol):
    name: str
    order: int
    def apply(self, f: FormProto) -> FormProto: ...
    def bracket_with(self, other: FunctionProto) -> FunctionProto: ...

@dataclass(frozen=True)
class Receipt:
    input_hash: str
    function_name: str
    order: int
    output_hash: str
    policy_version: str
    grade: str

_JOURNAL: list[Receipt] = [] # Prototype --- real impl uses append-only store

def bracket(
    f: FormProto,
    phi: FunctionProto,
    order: int = 3,
    grade: str = "Verification",
    policy_version: str = "1.0.0",
) -> FormProto:
    if order not in (1, 2, 3):
        raise ValueError(f"BCH order must be 1, 2, or 3; got {order}")

    # Phase: execute the truncated bracket
    r1 = phi.apply(f)
    if order == 1:
        out = r1
    else:
        r2 = _commutator(f, r1)
        if order == 2:
            out = r2
        else: # order == 3
            r3a = _commutator(f, r2)
            r3b = _commutator(r1, r2)
            out = _add(r3a, r3b)

    # Phase: validate closure (elided here)
    # Phase: compute provenance
    new_provenance = sha256(
        f"{f.content_hash}|{phi.name}|{order}".encode()
    ).hexdigest()
    out = replace(out, provenance=f"sha256:{new_provenance}")

    # Phase: journal
    _JOURNAL.append(Receipt(
        input_hash=f.content_hash,
        function_name=phi.name,
        order=order,
        output_hash=out.content_hash,
        policy_version=policy_version,
        grade=grade,

```

```

    ))
    return out

def _commutator(a: FormProto, b: FormProto) -> FormProto:
    # Implemented per algebra type; the kernel dispatches on schema_version.
    ...

def _add(a: FormProto, b: FormProto) -> FormProto:
    ...

```

This skeleton satisfies invariants I-B1 (same inputs $\rightarrow$ same outputs via deterministic `phi.apply`), I-B2 (BCH capped at 3), and I-B3 (journal append happens before return). A production implementation replaces the in-memory `_JOURNAL` with a proper append-only store and adds the 2PC protocol from §5.3.

14 Document Status

- **Version.** 1.0 (Initial PDR)
 - **Status.** Design review — open for technical critique
 - **Next review.** After Phase 0 exit criterion
 - **Author.** Bradley Wallace
 - **Document ID.** ACS Deterministic AI Stack PDR
-

End of document.

Amendment: scope and evidence, 24 September 2026

Deterministic replay and provenance are engineering properties of a specified implementation. They do not establish that its geometric features identify a unique physical source. The action audit supplies a concrete counterexample: different complete actions can share neutral observations while differing in transverse stability. A deployed witness must therefore retain its measurement convention, uncertainty and alternate-model controls. The release checks verify manuscript integrity and selected calculations; they do not certify production security or benchmark generalization.

For the complete-action and microscopic-carrier audit, see the companion papers *Complete Scalar Actions and Identifiability Boundaries in the ACS Framework* and *Condensate Binding, Charge Protection, and the Boundary of Particle Identification* in this repository's `papers/updates/` directory. The dated publication record separates fresh portable checks from preserved archival results; it does not claim a complete rerun of every historical experiment or a proof of RH.